

Resumen para el segundo parcial

Para el segundo parcial, entran las practicas 6 , 7 , 8 , 9 y 10

Resumen Practica 6 - Parametros:

Parametro Real (argumento)

- Es el valor que se le pasa cuando se **llama/invoca** a la rutina

```
saludar("Gianca");  
// Gianca es el parametro real (argumento)
```

Parametro Formal:

- Es el nombre que se usa en la definicion de una funcion o metodo
- Actua como variable local dentro de esa funcion

```
public void saludar (String unNombre)  
{  
    System.out.println("Hola " + unNombre);  
}  
// unNombre es el parametro formal.
```

Conceptos de pasajes de parametros IN - OUT - IN/OUT

Los conceptos segun el modo de pasaje de parametros son:

- Parametros IN: El parametro formal recibe el dato/valor desde el parametro real
- Parametros OUT: El parametro formal envia el dato al parametro real
- Parametros IN/OUT: El parametro formal recibe el dato del parametro real y el parametro formal le envia el dato al parametro real.

Practica pasajes de parametros

Modo IN:

El parametro formal recibe el dato a partir del parametro real.

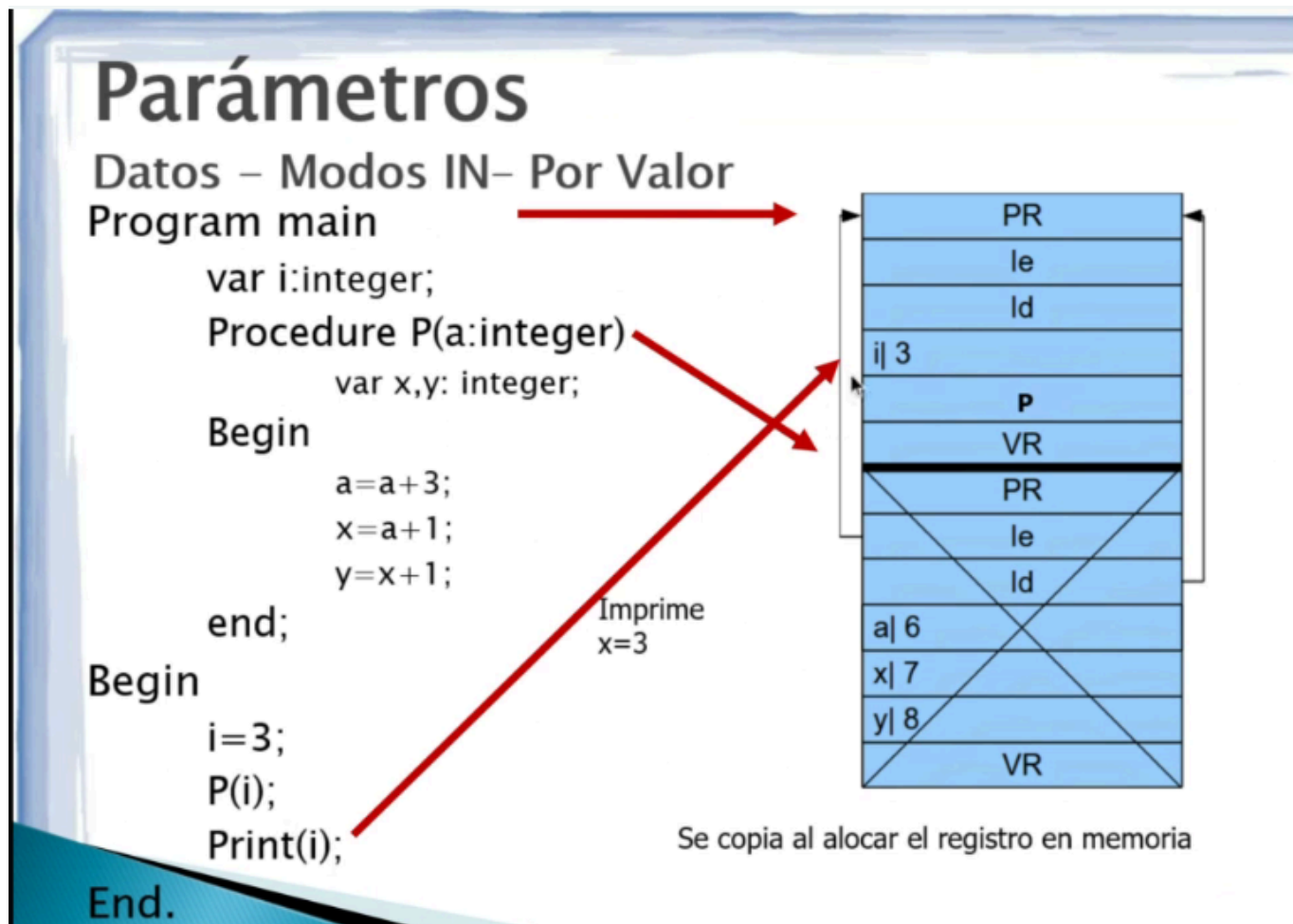
La conexión entre el parámetro real y el parámetro formal se hace al inicio, se copia el dato y luego corta la vinculación.

Pasaje IN por Valor:

El parámetro real se usa para inicializar el correspondiente parámetro formal.
Se copia el dato real.

Al pasar por parámetro, se copia el valor del dato, pero **si dentro de la rutina se modifica el parámetro real, solamente se vera afectado dentro de la rutina.**

Por lo que, al desalocar la rutina, el parámetro formal sigue teniendo el mismo valor antes de invocar a la rutina, ya que se le pasia una **copia del dato**.



Pasaje IN por valor constante:

Se envia un valor, pero la rutina **NO** puede modificarlo. Es decir, queda con un valor fijo que **no se puede cambiar**., y dependera del lenguaje si se realiza o no se realiza la copia.

La implementación **DEBE** verificar que el parámetro real **NO** sea modificado.

Modo OUT:

Se invoca a la rutina y cuando esta termina, devuelve el parametro formal al parametro real. **La conexion es al final**

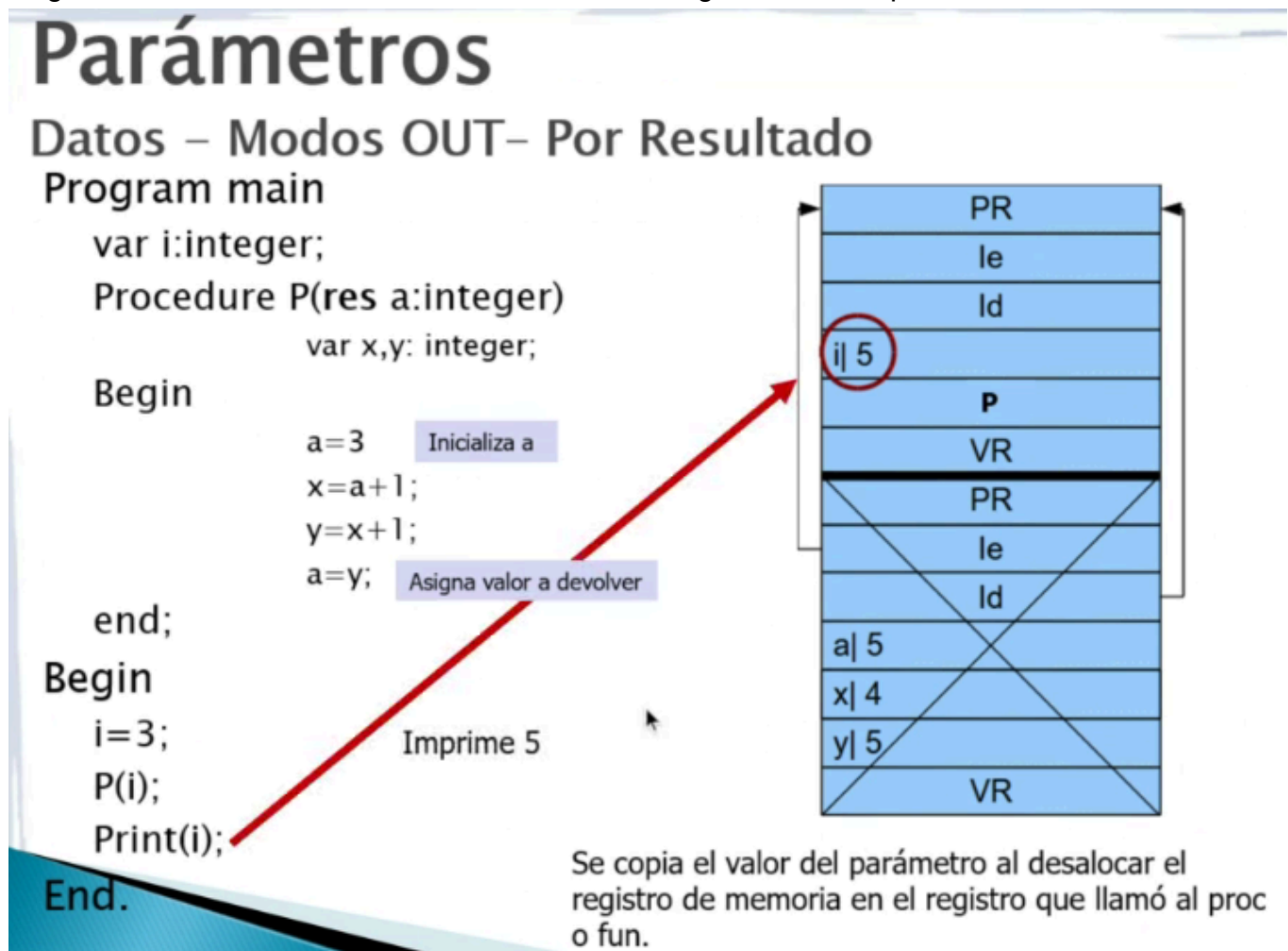
IMPORTANTE: En este modo de pasaje de parametro, la rutina **está obligada a asignarle un valor antes de que termine la rutina.**

Si la rutina NO asigna un valor antes de que termine, se considera un error.

Pasaje OUT, por resultado:

El valor del parametro formal de la rutina se copia al parametro real al **terminar la ejecucion de la rutina.**

El parametro formal es una variable sin valor inicial, porque se espera que en la rutina se le asigne un valor. Por ende, debemos inicializar de alguna forma el parametro real.



Modo IN / OUT:

La conexion se realiza al inicio y al final.

Pasaje IN / OUT por valor_resultado:

El parametro formal es una variable local que recibe una copia a la entrada del contenido del parametro real y a la salida el parametro real recibe una copia de lo que tiene el parametro formal. Una vez que termina de ejecutar el parametro formal le devuelve un valor al parametro real y lo copia.

Parámetros

Modos IN/OUT– Por Valor/Resultado

Program main

```

var i:integer;
Procedure P(in-out a:integer)
    var x,y: integer;

    Begin
        a=4
        x=a+1;
        y=x+1;
        a=y;
    end;

Begin
    i=3;
    P(i);
    Print(i);

End.
```

Se copia al alojar el registro y se modifica el parámetro formal al finalizar la ejecución de la rutina.

Pasaje IN / OUT por Referencia:

No se le hace una copia, si no que se le pasa la referencia (direccion de memoria / L-Valor) del parametro real al parametro formal.

La conexion es al inicio y **permanece** hasta el final.

Cualquier cambio que se realice en el parametro formal dentro del subprograma, quedara registrado en el parametro real, el cambio será automatico.

Parámetros

Datos – Modos IN/OUT– Por Referencia

Program main

```
var i:integer;
```

```
Procedura P(var a:integer)
```

```
var x,y: integer;
```

```
Begin
```

```
  a=4
```

```
  x=a+1;
```

```
  y=x+1;
```

```
  a=y;
```

```
end;
```

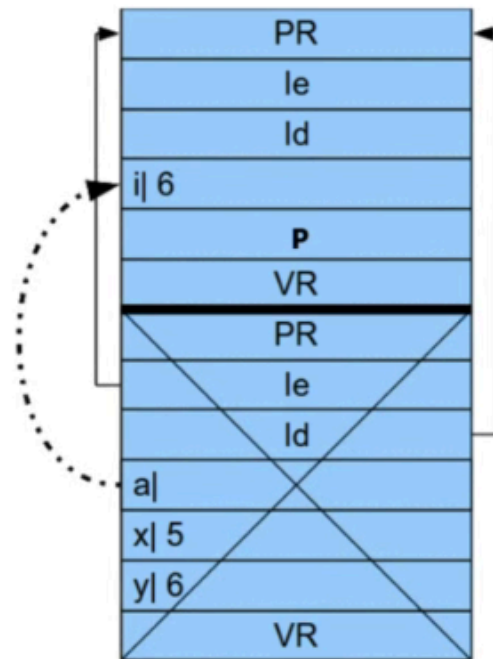
```
Begin
```

```
  i=3;
```

```
  P(i);
```

```
  Print(i);
```

```
End.
```



Se trabaja directamente sobre la variable referenciada

Por Referencia también es conocido por Variable

Pasaje IN / OUT por Nombre:

El parametro formal es sustituido "textualmente" por una expresion del parametro real.

Es importante saber que cada vez que se utilice el parametro real, se evaluará la expresion que recibió.

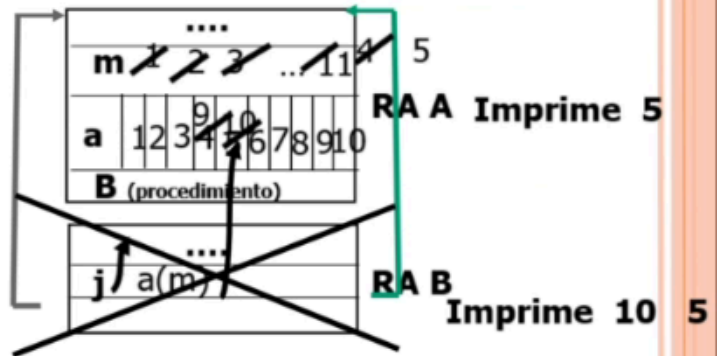
Por ejemplo si le pasamos al momento de invocarla: rutinaUno(x + 1)

Dentro de la rutina, cada vez que se use el parametro real, se evaluará x+1.

Parámetros

Datos – Modos IN/OUT–
Por Nombre

```
Procedure A ();  
  var m:integer;  
  a: array[1..10] of integer;  
Procedure B (nombre j:integer);  
  begin  
    j:=j+5;  
    m:=m+1;  
    j:= j+ m;  write (j,m);  
  end;  
begin  
  for m=1 to 10 begin  a[m]=m;  end;  
  m:=4;  B(a[m]);  
  write (m);  
end;
```



**Es fundamental preguntarse
¿qué significa "a(m)" en el
registro de activación del
"llamante". Este es el secreto del
parámetro por nombre.
Calcula la dirección durante la
ejecución**

54

Pasaje de parametros con DEEP y SHALLOW:

SHALLOW (parecido a buscar una variable por cadena dinámica): El ambiente de referencia, es el del subprograma que tiene el parámetro formal subprograma.

DEEP (parecido a buscar una variable por cadena estática): El ambiente es el del subprograma dónde está declarado el subprograma usado como parámetro real. Se utiliza en los lenguajes con alcance estático y estructura de bloque.

Puntos claves de esta practica:

Entender en que momento se realiza la conexion entre el parametro formal y el parametro real, ademas debe quedar en la pila de parametros todos los valores por los que pasó una variable.

Es importante saber que pasajes se hacen mediante referencia, cuales se hacen mediante valor, y en que momento se copia el valor del parametro real al parametro formal.

Si usamos pasaje de parametros por modo out, la rutina **ESTÁ OBLIGADA A ASIGNARLE UN VALOR AL PARAMETRO REAL.**, si no lo hace se tomaria como un error.

Si usamos pasaje de parametro por modo in/out por nombre, debemos tener en cuenta que cada vez que se use el parametro real dentro de la rutina, se evaluará la expresion que le mandemos. Ademas, segun el tipo de expresion podria generarse errores.

Deep y Shallow: Para mi es lo mismo que veniamos haciendo antes con cadena estatica y cadena dinamica, nada mas que le ponen otro nombre.

Resumen practica 7 - Sistemas y tipos de datos:

Que es un sistema de tipos y cual es su principal funcion?

Un sistema de tipos es un componente fundamental de un lenguaje de programacion que se encarga de asignar un tipo a los diferentes valores y expresiones dentro del programa.

La principal funcion de un sistema de tipos es verificar y garantizar la consistencia de los tipos de datos utilizados en el codigo, tanto en tiempo de compilacion como en tiempo de ejecucion. Esto ayuda a prevenir errores como intentar sumar un numero con una cadena o llamar a una funcion con un numero incorrecto de argumentos.

Tipado Fuerte vs Tipado Debil:

Tipado Fuerte:

impone reglas estrictas sobre como se pueden usar los tipos de datos. No permite realizar operaciones entre tipos incompatibles sin conversion explicita.

```
a = "5"
b = 3
print(a + b) # Error: no se puede sumar string y entero
```

se debe hacer una conversion explicita.

```
print(int(a) + b) # Salida: 8
```

Tipado Debil:

Permite mezclar tipos de datos con mas flexibilidad, haciendo conversiones automaticas (implicitas) cuando sea posible.

```
let a = "5";
let b = 3;
console.log(a + b); // Salida: "53" (JavaScript convierte el número a string)
```

Tipado estatico:

La verificacion de tipos se realiza en tiempo de compilacion. Los errores de tipo se detectan antes de ejecutar el programa

Un ejemplo de tipado estatico es Java, se debe declarar el tipo de cada variable.

```
int x = 30;
```

Tipado dinamico:

La verificacion de tipos ocurre en tiempo de ejecucion. Los tipos no se asocian a las variables sino a los valores.

Por ejemplo Python tiene tipado dinamico, las variables no se declaran con un tipo fijo, sino que Python permite que el tipo de una variable pueda cambiar dinamicamente.

```
x = 10  
x = "hola"
```

Definicion de tipo de dato:

Un tipo de dato es una categoria de valores que define que tipo de dato puede almacenar una variable y que operaciones se pueden realizar sobre ella.

Que es un tipo de dato definido por el usuario?

Un tipo definido por el usuario es aquel que se crea a partir de tipos ya existentes, generalmente para representar estructuras mas complejas o personalizadas. Permiten agrupar distintos datos en una sola entidad logica.

Definicion de: Producto cartesiano, correspondencia finita, union y recursion.

- Producto cartesiano: Es una combinacion de varios tipos de datos en una sola estructura, donde cada campo tiene un valor. Los registros o clases son ejemplos de tipo de producto cartesiano.
- Correspondencia finita: Es una estructura que asocia un conjunto finito de claves con valores. Tambien se conocen como diccionario, mapa o tabla hash.
- Uniones: Representa un tipo de dato que puede ser uno entre varios posibles tipos o valores.
- Tipos recursivos: Son tipos de datos que se definen en terminos de di mismos, permitiendo estructuras como arboles.

Manejo de punteros en C

Esto suelen preguntarlo en algunas preguntas de V o F

Que problemas existen en el manejo de punteros? Ejemplificar.

El manejo de punteros en C puede generar problemas si no se usan correctamente.

Por ejemplo pueden generarse errores como:

- Acceso no valido a memoria (punteros colgantes):
Ocurren cuando un puntero apunta a una direccion de memoria que ya no es valida.

```
int* ptr;  
*ptr = 5; // ERROR: ptr no apunta a una dirección válida
```

- Doble liberacion de memoria:
Liberar dos veces la misma direccion de memoria puede producir comportamientos indefinidos.

```
int* p = malloc(sizeof(int));  
free(p);  
free(p); // ERROR: doble liberación
```

Que es un TAD? que características debe cumplir para que se considere un TAD?

Un tipo abstracto de datos (TAD) es una estructura que define un tipo de dato junto con un conjunto de operaciones validas sobre ese dato, sin exponer su implementacion interna.

Para que una unidad sea considerada un TAD, debe cumplir con las siguientes características:

- Encapsulamiento: Los datos internos estan ocultos y solo pueden ser manipulados mediante operaciones definidas (metodos).
- Interfaz definida: Se especifica que operaciones estan disponibles y como se usan sin detallar como estan implementadas.

Resumen Practica 8 - Estructuras de control y Sentencias

Sentencias simples vs Sentencias compuestas:

Sentencia simple: Es una instruccion que realiza una accion especifica. Por ejemplo:

```
int x = 5 + 10;
```

Sentencia compuesta: Es un bloque de sentencias agrupadas entre llaves { }, tratadas como una sola unidad logica.

```
if (x > 5) {  
    System.out.println("x es mayor que 5");  
    x = x + 1;  
}
```

Analizar como se implementa la asignacion en C.

Una restriccion de como se implementa la asignacion en C, es que del lado izquierdo de la asignacion, debe tener si o si un L-Valor (una variable), no puede tener un R-Valor(resultado de una expresion.)

¿Una expresión de asignación puede producir efectos laterales que afecten al resultado final, dependiendo de cómo se evalúe? De ejemplos.

Si, una expresion de asignacion en C, puede producir efectos secundarios que influyen en el resultado final de un programa, dependiendo del contexto en que se evalúe.

```
a = b = c = 0;
```

Que significa que un lenguaje utilice circuito largo o corto para evaluar una expresion?

Circuito corto: La evaluacion de una expresion logica (con AND o OR) se detiene tan pronto como se conozca el resultado de la evaluacion.

Los operandos se evaluan de izquierda a derecha, y el resultado se puede determinar con los primeros operandos, los restantes no se evaluan.

Ejemplo:

```
if (x > 0 && esPositivo(x)) {  
    System.out.println("x es positivo");  
}
```

Si $x < 0$, la segunda condicion no se evaluaria por el circuito corto, dado que la primer condicion es falsa, las condiciones que le siguen no las evaluan. Corta el circuito lo antes posible.

Circuito largo: Evalua todos los operandos, incluso si ya se sabe el resultado.

Ejemplo:

```
if (x > 0 & esPositivo(x)) {  
    System.out.println("x es positivo");  
}
```

```
}
```

En este ejemplo, esPositivo(x) se evalúa SIEMPRE dado al circuito largo.

Conclusiones y diferencias:

Por lo menos en Java, con los ejemplos que vimos:

- En el circuito corto se usa el doble operador lógico. (&&)
- En el circuito largo se usa un solo operador lógico. (&)

Estructura de repetición en Pascal y ADA

<pre>..... var i, z:integer; Procedure A; begin i:= i +1; end; begin z:=5;</pre>	<pre>for i:=1..5 do begin z:=z*5; A; z:=z + i; end; end;</pre>
--	--

Analice en las versiones estándar de ADA y Pascal, si este código puede llegar a traer problemas. Justifique la respuesta.

Pascal: tiene restricciones sobre la modificación de la variable iteradora `i` que se usa en el `for`.

El estándar de Pascal establece que la variable iteradora no puede ser modificada por ejemplo en este caso dentro del bucle, cuando se llama al `procedure A`, intenta incrementar el valor de `i`, esto puede llevar a un comportamiento indefinido.

Otra cosa a tener en cuenta es que el valor de la variable '`i`', ni bien termina el `for`, se considera con valor indefinido en Pascal estándar. Esto significa que no se puede predecir con seguridad que valor tendrá '`i`' después de la ejecución del bucle.

ADA: La variable iteradora del bucle (`i` en este caso) se declara implícitamente dentro del bucle y su alcance se limita al bucle.

NO permite la modificación de la variable dentro del bucle.

En el código que nos dan, `i` es una variable global, por lo que en ADA, está permitido pero se considera una mala práctica por razones de claridad y mantenimiento del código.

No generaría un error en tiempo de compilación por la modificación de la variable `i`, pero sí podría ser problemático en términos de legibilidad y mantenimiento.

Switch / Case y Break:

El switch es una estructura de control que permite comparar el valor de una variable con multiples opciones (case) y ejecutar el bloque de codigo correspondiente.

```
switch (variable) {  
    case valor1:  
        // código si variable == valor1  
        break;  
    case valor2:  
        // código si variable == valor2  
        break;  
    default:  
        // código si no coincide con ningún case  
}
```

El `break` es importante, ya que si no se pone, se siguen ejecutando los `case` siguientes. A esto se lo llama `fallthrough` (caída hacia abajo).

Posibles errores:

- Olvidar el `break`: Puede pasar que se ejecuten líneas de manera no intencionada, esto lo solucionamos poniendo la sentencia `break` para hacer que salga del `case` y no continúe ejecutando sentencias del `case`.
- No incluir un `default`: Puede generar comportamiento inesperado si la variable 'puntos' tiene un valor que no es manejado por ninguna sentencia del `case`.

Resumen Practica 9 - Excepciones

Que es una excepcion?

Una excepcion es una condicion inesperada o inusual que ocurre durante la ejecucion de un programa y no puede ser manejada en el contexto actual.

Las excepciones denotan un comportamiento indeseable pero es necesario controlarlo.

Una excepcion rompe el flujo normal de ejecucion y ejecuta un manejador de excepciones registrado previamente.

Qué debería proveer un lenguaje para el manejo de las excepciones? ¿Todos los lenguajes lo proveen?

Para que un lenguaje trate excepciones, debe proveer minimamente:

- Un modo de definir las
- Una forma de reconocerlas

- Una forma de lanzarlas y capturarlas

Modelo de manejo de excepciones

Los modelos de manejo de excepciones son:

Reasuncion	Terminacion
Maneja la excepcion y continua la ejecucion	Termina la ejecucion y maneja la excepcion
Se refiere a la posibilidad de retornar la ejecucion normal del programa despues de manejar una excepcion . El controlador de excepciones realiza las acciones necesarias para manejar la excepcion, y luego el programa continua su ejecucion a partir del punto donde se produjo la excepcion.	El controlador de excepciones realiza las acciones necesarias para manejar la excepcion, pero no se retorna al punto donde se produjo la excepcion , sino que continua su ejecucion a partir de la finalizacion del manejador.
Lenguajes que lo utilizan: PL / 1	Lenguajes que lo utilizan: ADA, C++, Java, Python, PHP

Resumiendo:

- **Modelo de terminacion:** NO pueden continuar en el punto donde se produjo la excepcion, se transfiere el control a un manejador y NO continua en el punto donde se produjo la excepcion.
- **Modelo de Reasuncion:** El programa puede continuar la ejecucion en el mismo punto donde se lanzo la excepcion, despues de ser manejada.

Como lanzar y manejar una excepcion

En Java:

La lanzamos con:

```
throw new IllegalArgumentException("Valor inválido");
```

La manejamos con bloques try / catch

```
try {  
    metodoRiesgoso();  
}
```

```
} catch (IllegalArgumentException e) {  
    System.out.println("Error: " + e.getMessage());  
}
```

En Python:

La lanzamos con:

```
raise ValueError("Dato incorrecto")
```

La manejamos con bloques try / except

```
try:  
    funcion_riesgosa()  
except ValueError as e:  
    print("Error:", e)
```

Puntos clave al momento de trabajar con estructuras repetitivas y excepciones:

Cuando tenemos un try-catch **dentro** del for

```
for (int i = 0; i < 5; i++) {  
    try {  
        System.out.println("Iteración " + i);  
        if (i == 2) {  
            throw new Exception("Error en i = 2");  
        }  
    } catch (Exception e) {  
        System.out.println("Capturada excepción: " + e.getMessage());  
    }  
}  
System.out.println("Fin del bucle");
```

✓ **Como el try-catch está dentro del for**: se captura la excepción por iteración, y el bucle continúa.

Cuando tenemos un try-catch **fuera** del for

```
try {  
    for (int i = 0; i < 5; i++) {  
        System.out.println("Iteración " + i);  
        if (i == 2) {  
            throw new Exception("Error en i = 2");  
        }  
    }  
}
```

```

    }
}
} catch (Exception e) {
    System.out.println("Capturada excepción: " + e.getMessage());
}
System.out.println("Fin del bucle");

```

❌ Como el `try-catch` está fuera del `for` : cuando ocurre una excepción, se sale del `for` inmediatamente, y ya no hay más iteraciones

Resumen Practica 10 - Paradigmas de Lenguajes de Programacion

Los paradigmas representan diferentes formas de pensar y organizar el código para resolver problemas. Cada paradigma se adapta mejor a ciertos contextos.

Lenguaje Funcional:

En un **lenguaje funcional**, un programa no se basa en una secuencia de instrucciones que modifica el estado de la memoria (como en los lenguajes procedurales.), sino se basan en la **evaluación de funciones y expresiones matemáticas**.

- Es una composición de funciones.
- No utilizan variables que cambian de valor.
- Se basa en el modelo matemático de funciones (como en cálculo lambda)
- Utiliza recursión en lugar de estructuras como bucles como `for` o `while`

,En un lenguaje funcional las funciones se definen en el entorno o ámbito de definición** (en inglés, *environment*).

Esto puede ser:

- Un **módulo** o archivo.
- Un **bloque de código** (como en una expresión `let` o `where` en Haskell).
- El **entorno global** si la función está al nivel principal del programa.

En lenguajes funcionales, una variable representa un **valor fijo (es inmutable.)**

A diferencia con los lenguajes imperativos, las variables **en los lenguajes funcionales NO cambian de valor**, mientras que en un lenguaje imperativo pueden cambiar de valor.

Que es una expresión en un lenguaje funcional?

Una expresión en un lenguaje funcional, es una combinación de funciones, valores, operadores y variables que pueden evaluarse para producir un resultado.

```
numeros.stream()
    .filter(n -> n > 10)           // Filtra los mayores que 10
    .map(n -> n * 2)               // Multiplica por 2
    .forEach(System.out::println); // Muestra el resultado
```

El valor de una expresion depende de sus argumentos (entradas), no depende del estado del programa ni de cambios en la memoria.

Siempre devuelve el mismo resultado para los mismos valores (esto se llama transparencia referencial).

Los lenguajes funcionales suelen usar evaluacion perezosa (lazy evaluation), aunque algunos usan evaluacion estricta (strict evaluation).

- Evaluacion perezosa: Una expresion solo se evalua cuando se necesita su valor.
- Evaluacion estricta: Las expresiones se evaluan tan pronto como sean definidas.

Conclusion: La evaluacion perezosa permite trabajar con estructuras infinitas y mejorar la eficiencia evitando calculos innecesarios.

Un lenguaje funcional es fuertemente tipado?

Si, la mayoría de los lenguajes son **fuertemente tipados**, pero tambien hay algunos que son **estaticamente tipados**.

Los lenguajes funcionales modernos (como Haskell) son fuertemente tipados porque:

- Se basan en funciones puras, y los tipos ayudan a garantizar que las funciones reciban los argumentos correctos y devuelvan resultados validos.
- El chequeo de tipos ayuda a prevenir errores comunes, ya que muchas veces los errores de logica o mal uso de funciones se detectan en **tiempo de compilacion**.
- Los lenguajes funcionales buscan ser matematicamente correctos y previsibles, y el uso fuerte de tipos refuerza esto.

Conclusion: Los tipos fuertes en la programacion funcional permiten mayor seguridad, legibilidad y correccion matematica.

Paradigma Orientado a Objetos

Un programa escrito en **Programación Orientada a Objetos (POO)** se basa en **objetos que representan entidades del mundo real**, los cuales tienen **estado (atributos)** y **comportamiento (métodos)**.

El programa se estructura como una **colección de clases** que definen el molde para crear esos objetos.

Cuales son los elementos mas importantes de la programacion orientada a objetos?

- **Clases:** Son plantillas o modelos que definen atributos y métodos.
- **Objetos:** Instancias de clases que encapsulan datos y comportamientos.
- **Encapsulamiento:** Oculta los detalles internos del objeto, exponiendo solo lo necesario.
- **Herencia:** Permite que una clase herede características de otra.
- **Polimorfismo:** Objetos diferentes pueden responder al mismo mensaje de manera distinta.
- **Abstracción:** Permite representar entidades complejas mediante modelos simplificados.

La posibilidad de ocultamiento y encapsulamiento para los objetos es el primer nivel de abstracción de la POO, ¿cuál es el segundo?

- El primer nivel es el encapsulamiento (ocultar datos internos)
- El segundo nivel es la herencia, que permite modelar jerarquias y reutilizar código, definiendo clases generales y derivadas mas específicas.

Paradigma Logico:

El paradigma logico es un estilo de programacion donde no se indica como se debe hacer algo, sino que se **describe que es lo que se quiere lograr** , usando hechos, reglas y consultas.

Características:

- Se describe el problema con **hechos y reglas**
- Se hace una **consulta (pregunta)** y el sistema trata de **probar** si puede deducir una respuesta lógica
- El lenguaje más conocido de este paradigma es **Prolog**

```
% Hechos:
padre(juan, maria).
padre(juan, jose).

% Regla:
hermano(X, Y) :- padre(P, X), padre(P, Y), X \= Y.

% Consulta:
?- hermano(maria, jose).
```

Que representa una variable en el paradigma logico?

Las variables son valores desconocidos que el sistema intentara encontrar para satisfacer una consulta. Se escriben con mayuscula (ejemplo: X , Y , Padre)

Que es una constante en el paradigma logico?

Una constante es un valor concreto que NO cambia.

Se escriben en minuscula en Prolog (ejemplo: `juan` , `maria` , `rojo`)

Ejercicio de paradigmas:

Teniendo en cuenta el siguiente problema, se lee una variable entera por teclado y si es par se imprime “El valor ingresado es PAR” y si es impar imprime “El valor ingresado es impar”, implemente este ejemplo en cada uno de los paradigmas presentados en esta práctica.

◆ Imperativo (Pascal):

```
program ejercicio13;
var
  numero : integer;
begin
  readln(numero);
  if (numero MOD 2 = 0) then
    writeln('El numero',numero,' es PAR')
  else
    writeln('El numero',numero,' es IMPAR');
end.
```

◆ Funcional (Haskell):

```
main = do
  input <- getLine
  let n = read input :: Int
  putStrLn (if even n then "El valor ingresado es PAR" else "El valor
ingresado es impar")
```

◆ Orientado a objetos (Java):

```
public class Ejercicio13 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        if (n % 2 == 0)
            System.out.println("El valor ingresado es PAR");
        else
            System.out.println("El valor ingresado es impar");
    }
}
```

◆ Lógico (Prolog):

```
par(N) :- 0 is N mod 2.
impar(N) :- 1 is N mod 2.

mensaje(N) :- par(N), write('El valor ingresado es PAR').
mensaje(N) :- impar(N), write('El valor ingresado es impar').

% Consulta:
% ?- mensaje(4).
```

Características de lenguajes basados en Scripts.

- Interpretados:
Los scripts no necesitan compilarse. Son ejecutados por un interprete, lo que permite una ejecucion inmediata y facilita el desarrollo rapido.
- Facilidad de uso:
Estan diseñados para ser simples de escribir y entender, con una sintaxis amigable y menor cantidad de codigo necesario para realizar tareas comunes.
- Automatizacion:
Se usan frecuentemente para automatizar tareas repetitivas, especialmente en entornos como administracion de sistemas, desarrollo web o pruebas automaticas.
- Tipado dinamico:
No se necesita declarar el tipo de las variables. El tipo se determina en tiempo de ejecucion.
Lenguajes comunes que utilizan scripts:
- **Python** 
- **JavaScript** (tanto en frontend como en backend con Node.js)
- **Bash** (Shell scripting en Linux/Unix)
- **Ruby**
- **PHP**
- **Perl**
- **Lua**
- **PowerShell** (en entornos Windows)

En general, los lenguajes basados en scripts utilizan tipado dinamico.

- El tipado dinamico se determina automaticamente en tiempo de ejecucion.

```
x = 5
```

```
x = "Hola"
```

Resumen de constructores:

- Producto cartesiano:
 - Este constructor crea un tipo cuyos elementos son n-tuplas ordenadas, donde cada componente pertenece a un conjunto diferente.
 - Un ejemplo de alguna estructura en un código puede ser la estructura struct / registros, donde se definen campos y cada campo permite un tipo de dato distinto.
- Correspondencia Finita
 - Define una función de un conjunto finito de valores de un tipo de dominio DT a valores de un tipo de rango RT
 - Un ejemplo de alguna estructura en un código puede ser el subrango.
 - Arreglos (arrays): Son una forma de correspondencia finita, representan una colección de elementos donde se accede a cada uno mediante un índice que es una correspondencia finita del dominio (los índices).
- Union y Union Discriminada
 - Define un tipo como la disyunción de dos o más tipos dados, lo que permite la manipulación de diferentes tipos en distintos momentos durante la ejecución.
 - Un ejemplo en código puede ser un enum
- Recursion:
 - Un tipo de dato recursivo T se define como una estructura que puede contener componentes del tipo T.
 - Un ejemplo en código es un puntero, una variable puntero cuyo r-valor es una referencia a un objeto.

Como darnos cuenta rapido de que constructor es?

- Si vemos una clase (en java), un registro (en pascal), podemos decir que es producto cartesiano, ya que es un conjunto de n tuplas ordenadas donde cada componente pertenece a un conjunto diferente.
- Si vemos un método (en java) que devuelve un valor, podemos decir que es correspondencia finita, ya que devuelve un conjunto finito de valores dependiendo de su entrada (parámetro)
- Si vemos un array (en java o Pascal), o tupla (en Python) podemos decir que es correspondencia finita ya que están formados por un conjunto finito de valores
- Si vemos una clase (en java) que tenga una instancia de la misma clase, (suelen poner en código la clase de árbolBinario, donde tiene de la misma instancia un Hijo izquierdo y un Hijo derecho) podemos decir que es producto cartesiano ya que es un conjunto de n tuplas

ordenadas donde cada componente pertenece a un conjunto diferente y ademas que es de tipo recursivo dado que la clase T puede contener componentes del mismo tipo T (en la definicion deberiamos reemplazar T por el nombre de la clase que tenga recursion.)

Resumen de lenguajes compilados, interpretados, tipado fuerte, tipado debil, tipado estatico y tipado dinamico.

Posibles conceptos a evaluar en algun verdadero o falso

Tipo de dato: Es un conjunto de valores y un conjunto de operaciones que puede realizar el tipo de dato.

La union discriminada es mas segura que la union, debido a que con la union discriminada podemos saber el tipo concreto de un elemento en un momento dado, mientras que con la union sabemos que un elemento puede ser de varios tipos, pero no conocemos que tipo concreto es.

Un objeto se dice accesible si alguna variable en la pila lo apunta directa o indirectamente.

Un lenguaje, para que permita definir TADs debe proveer encapsulamiento y ocultamiento. es importante que tenga las dos, ya que no es suficiente solo con tener una de las dos propiedades.

Abstraer es representar algo descubriendo sus características esenciales y suprimiendo las que no lo son. El principio básico de la abstracción es la información oculta.

El proceso de construir nuevos tipos se llama abstracción de datos.

Si el lenguaje es fuertemente tipado, el compilador puede garantizar la ausencia de errores de tipo en los programas.

Un lenguaje se dice fuertemente tipado si el sistema de tipos impone restricciones que aseguran que no se produzcan errores de tipo en ejecución.

Un lenguaje se dice fuertemente tipado si todos los errores de tipo se detectan.